



Core Knowledge Series

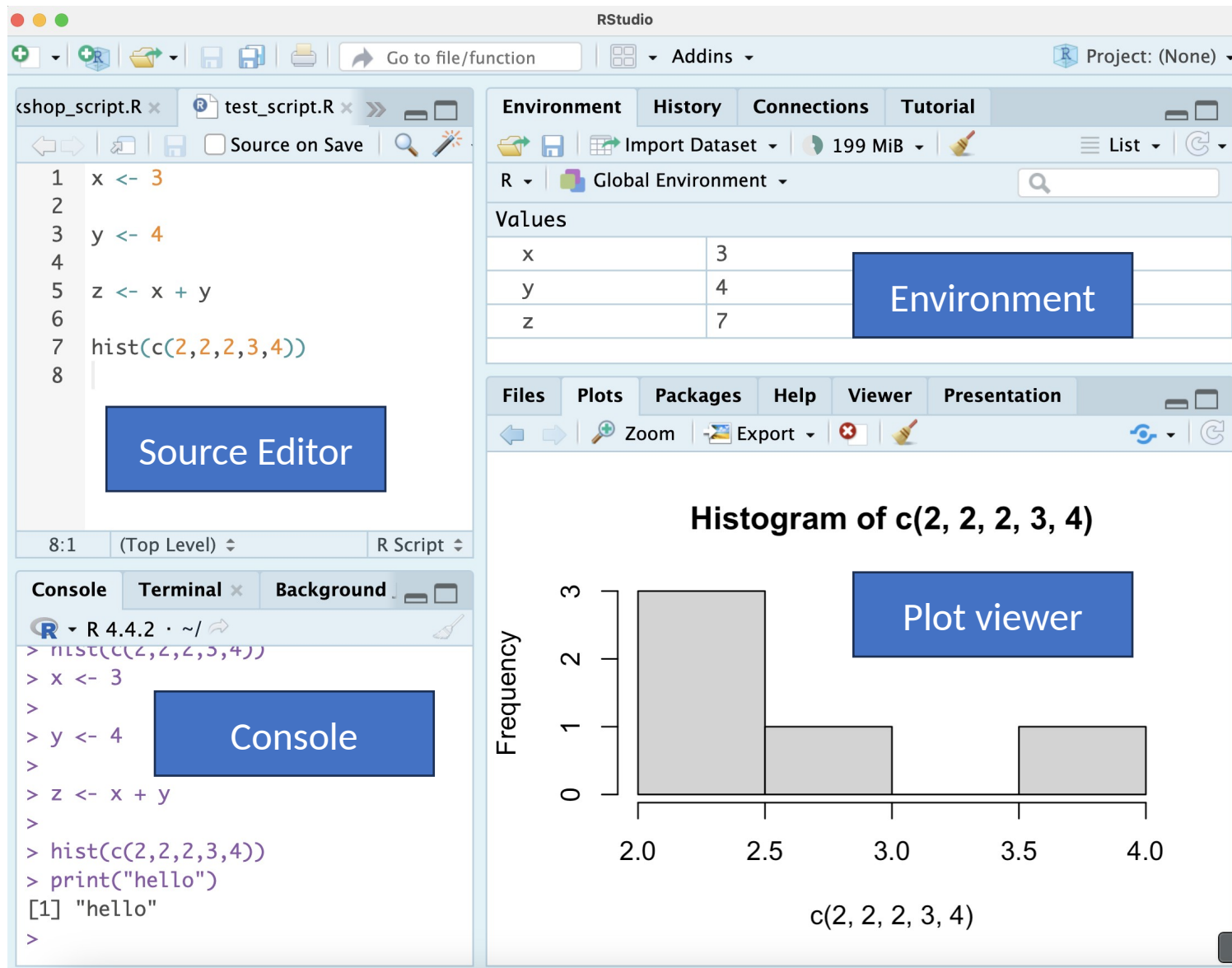
Hands-On with Seurat

Bioinformatics Core (BSR)

R Studio and R

- R is a powerful open-source programming language specifically designed for statistical computing, data analysis, and visualization. Many bioinformatics tools are developed in R.
- RStudio is user-friendly and makes coding in R easier with features like debugging, plotting, and environment.
- Packages native to R
 - Bioinformatics tools:
 - DESeq2 for RNA-Seq
 - Seurat for single-cell
 - Diffbind for ChIP and ATAC-seq
 - Data Manipulation
 - dplyr, tibble, tidyr, tidyverse

R Studio



- **Source Editor (Top-Left)**
 - Writing/editing R and Rmd scripts
- **Environment (Top-Right)**
 - Stores all created variables
- **Console/Terminal (Bottom-Left)**
 - Console: Executes R commands
 - Terminal: Navigate through files
- **Files/Plots/Help (Bottom-right)**
 - Mainly used for plot viewing. Can also navigate files and display help for functions.

Libraries in R

```
install.packages("Seurat")  
library(Seurat)
```

- **Libraries** in R are collections of functions, datasets, and documentation that extend R's capabilities.
- Libraries must be **installed** and **loaded** before use

Basic syntax – Variables and Functions

```
numbers <- c(1, 2, 3, 4, 5)
result <- sum(numbers)
print(result)
```

- **Object** – Fundamental data structure
 - Example: List/vector of numbers
- **Variable** – Name to store fundamental data structure
 - Example: “numbers” variable
- **Function** – Takes in input, performs operation(s), produces output
 - Sum function

Basic syntax – Variables and Functions

```
numbers <- c(1, 2, 3, 4, 5)
result <- sum(numbers)
print(result)
```

- Variables are assigned using “<-” in R
- Functions often come before parenthesis with **arguments** inside
 - Inputs to a function that tell provide specifications for the function to perform its task
- sum(numbers)
 - Numbers is the argument to the sum function.
- How can we see what arguments a function takes?

Using the help operator

?sum

sum {base}

R Documentation

Sum of Vector Elements

Description

sum returns the sum of all the values present in its arguments.

Usage

```
sum(..., na.rm = FALSE)
```

Arguments

... numeric or complex or logical vectors.

na.rm logical (TRUE or FALSE). Should missing values (including NaN) be removed?

Details

This is a generic function: methods can be defined for it directly or via the [Summary](#) group generic. For this to work properly, the arguments ... should be unnamed, and dispatch is on the first argument.

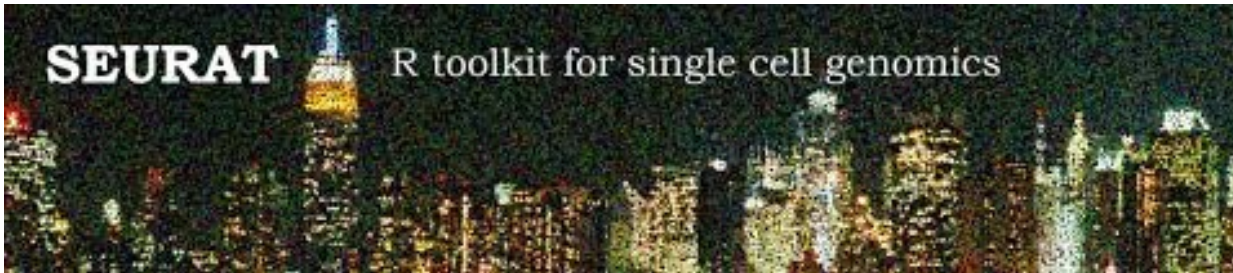
If na.rm is FALSE an NA or NaN value in any of the arguments will cause a value of NA or NaN to be returned, otherwise NA and NaN values are ignored.

Logical true values are regarded as one, false values as zero. For historical reasons, NULL is accepted and treated as if it were integer(0).

Loss of accuracy can occur when summing values of different signs: this can even occur for sufficiently long integer inputs if the partial sums would cause integer overflow. Where possible extended-precision accumulators are used, typically well supported with C99 and newer, but possibly platform-dependent.

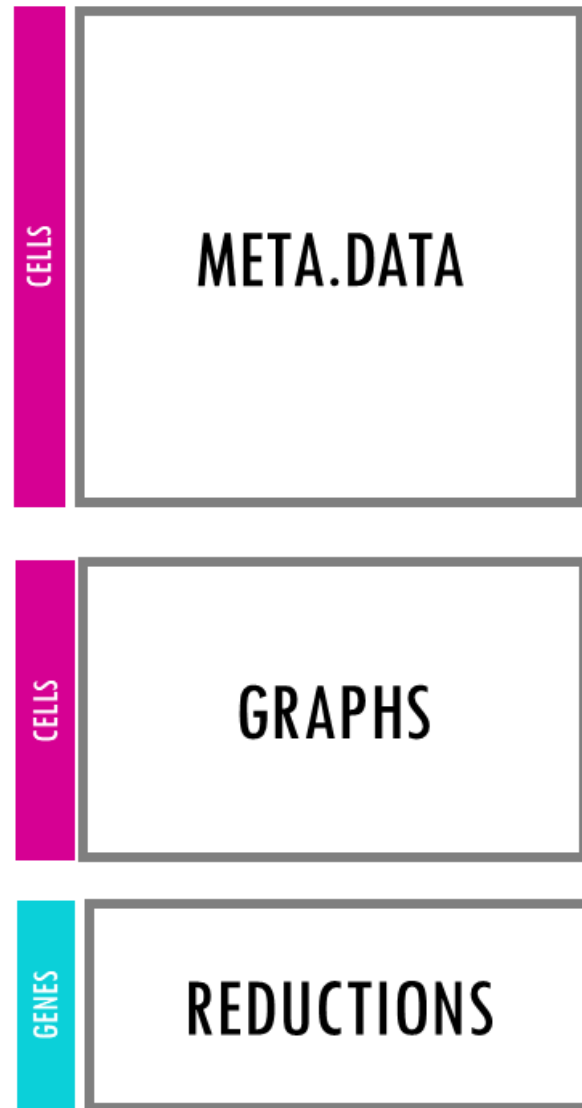
- The help operator "?" can be placed in front of a function to open documentation for that function
 - Example: ?sum
- Note: The sum function has an optional parameter na.rm that will remove NA values if you specify TRUE
 - Default is na.rm = FALSE
- The ? operator is very helpful for Seurat functions
 - Well documented and often have many arguments

Seurat

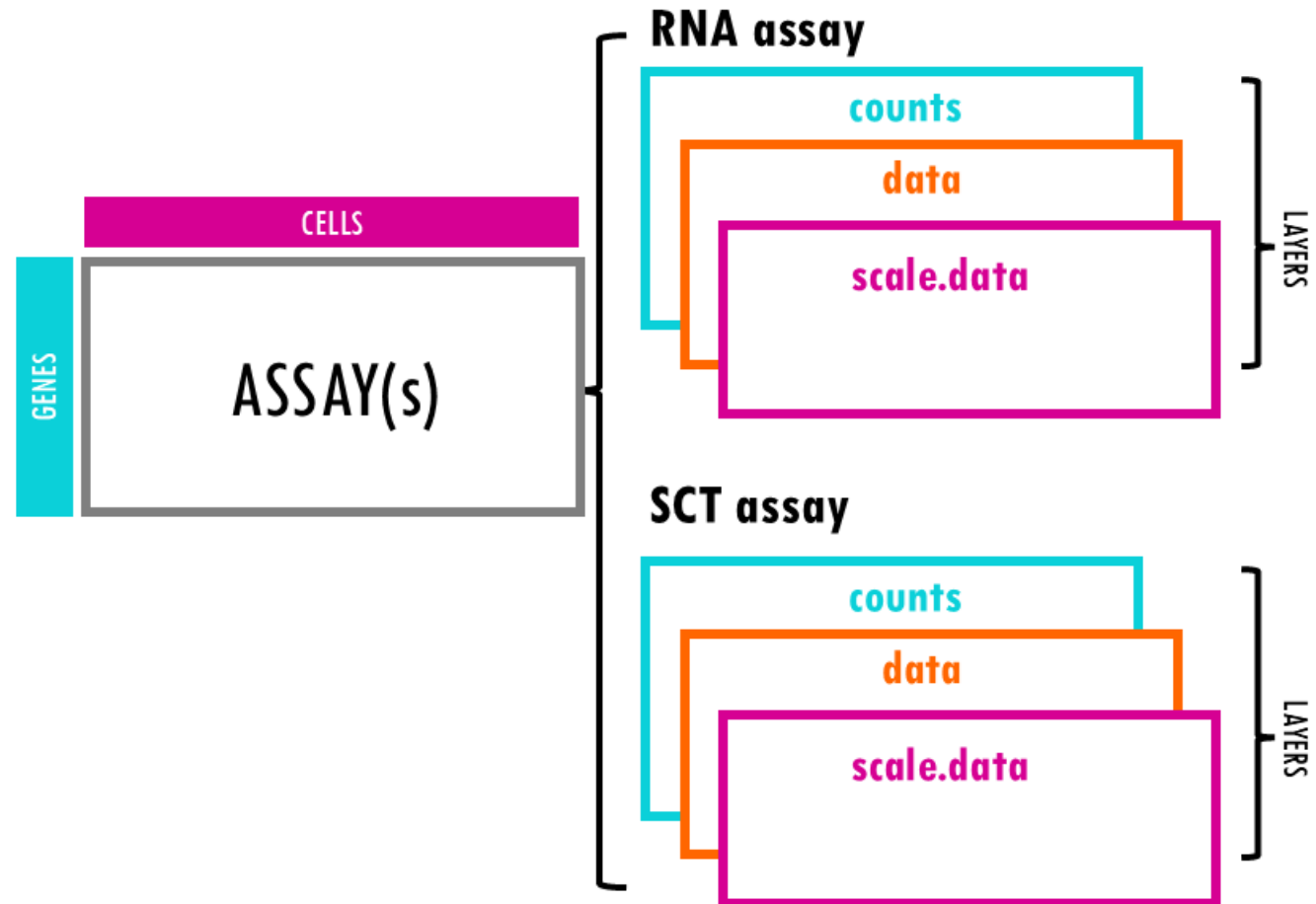


- **Seurat** is an R package for analyzing single-cell RNA-seq data.
- It provides functions for normalization, clustering, plotting, differential analysis, integration, etc.

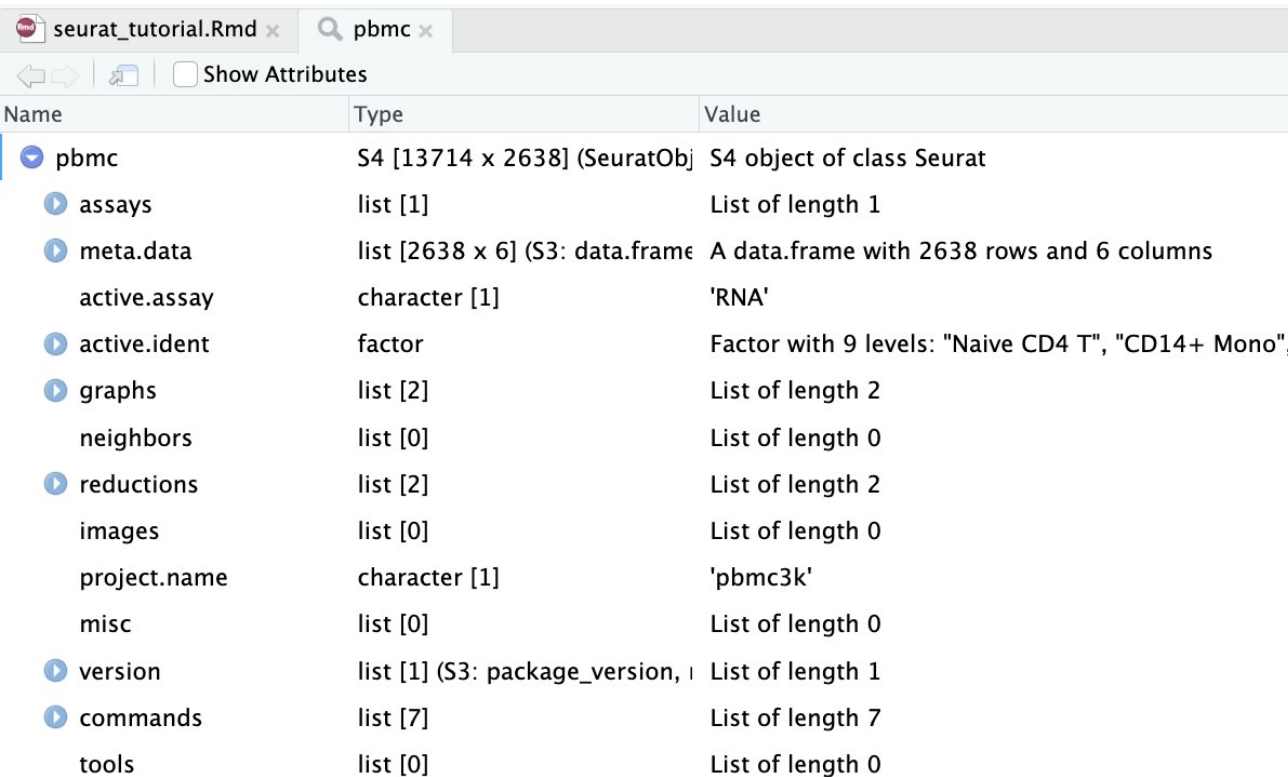
Seurat Object



SEURAT OBJECT



Our PBMC Seurat Object



The screenshot shows the RStudio interface with a file named 'seurat_tutorial.Rmd' and a search for 'pbmc'. Below the search bar, there is a 'Show Attributes' checkbox. The main table displays the structure of the 'pbmc' Seurat object, listing its components (assays, meta.data, active.ident, graphs, reductions, images, project.name, misc, version, commands, tools) and their respective types and values.

Name	Type	Value
pbmc	S4 [13714 x 2638] (SeuratObj)	S4 object of class Seurat
assays	list [1]	List of length 1
meta.data	list [2638 x 6] (S3: data.frame)	A data.frame with 2638 rows and 6 columns
active.assay	character [1]	'RNA'
active.ident	factor	Factor with 9 levels: "Naive CD4 T", "CD14+ Mono",
graphs	list [2]	List of length 2
neighbors	list [0]	List of length 0
reductions	list [2]	List of length 2
images	list [0]	List of length 0
project.name	character [1]	'pbmc3k'
misc	list [0]	List of length 0
version	list [1] (S3: package_version,)	List of length 1
commands	list [7]	List of length 7
tools	list [0]	List of length 0

- **Seurat** objects contain **slots**. Think of them as compartments of the object.
 - Assays: Holds expression data
 - counts
 - data
 - scale.data
 - meta.data: Per-cell metadata
 - active.ident: Current identity of cells
 - Usually cluster number or cell type
 - reductions: Dimensionality reductions
 - PCA, tSNE, UMAP

R syntax with Seurat Object

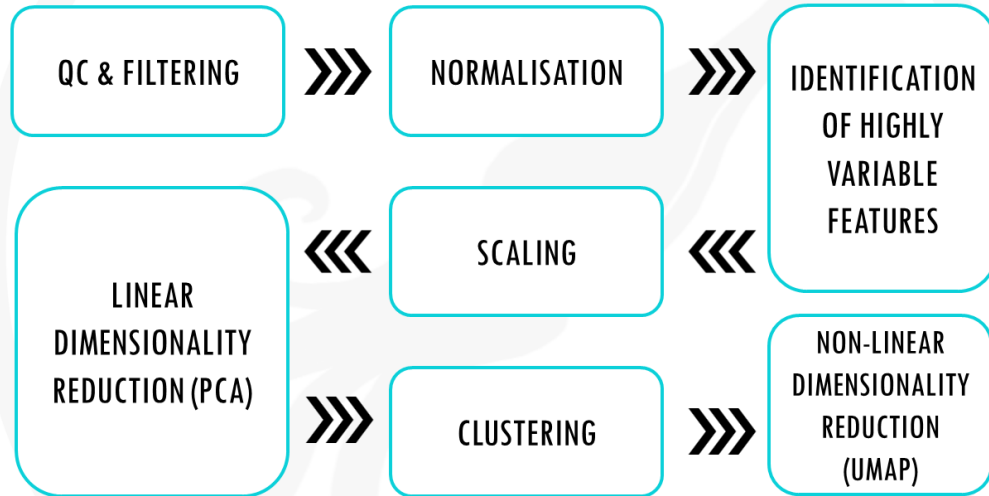
- `$` operator is used to access metadata.
- `@` operator is used to access other slots

```
▼ meta.data      list [2638 x 6] (S3: data.frame) A data.frame with 2638 rows and 6 columns
  orig.ident      factor                Factor with 1 level: "pbmc3k"
  nCount_RNA      double [2638]         2419 4903 3147 2639 980 2163 ...
  nFeature_RNA     integer [2638]       779 1352 1129 960 521 781 ...
  percent.mt       double [2638]        3.02 3.79 0.89 1.74 1.22 1.66 ...
  RNA_snn_res.0.5  factor                Factor with 9 levels: "0", "1", "2", "3", "4", "5", ...
  seurat_clusters  factor                Factor with 9 levels: "0", "1", "2", "3", "4", "5", ...
```

```
> head(pbmc$percent.mt)
AAACATACAACCAC-1 AAACATTGAGCTAC-1 AAACATTGATCAGC-1 AAACCGTGCTTCCG-1 AAACCGTGTATGCG-1 AAACGCACTGGTAC-1
      3.0177759      3.7935958      0.8897363      1.7430845      1.2244898      1.6643551
```

```
> pbmc@assays
$RNA
Assay (v5) data with 13714 features for 2638 cells
Top 10 variable features:
 PPBP, LYZ, S100A9, IGLL5, GNLY, FTL, PF4, FTH1, GNG11, S100A8
Layers:
 counts, data, scale.data
```

Seurat workflow



- **Create object** – Read10X, CreateSeuratObject
QC & filter – % mito, nFeature/nCount, subset
- **Normalize** – NormalizeData() or SCTransform()
- **Variable features** – FindVariableFeatures()
- **Scale data** – ScaleData() PCA – RunPCA(),
inspect PCs (ElbowPlot)
- **Clustering** – FindNeighbors(), FindClusters()
UMAP/tSNE – RunUMAP() / RunTSNE()
- **Find markers** – FindMarkers(),
FindAllMarkers()
- **Annotate** – label clusters with known markers
- **Save object** – saveRDS()

Creating the Seurat Object

```
```\r}\nlibrary(Seurat)\nlibrary(dplyr)\n\n# Creating folder and reading in data\noutdir <- "~/single_cell_practice"\noutdir <- path.expand(outdir)\nif (!dir.exists(outdir)) {\n  dir.create(outdir, recursive = TRUE)\n}\n\ndata_file <- file.path(outdir, "pbmc3k_filtered_gene_bc_matrices.tar.gz")\ndownload.file(\n  url = "https://cf.10xgenomics.com/samples/cell/pbmc3k/pbmc3k_filtered_gene_bc_matrices.tar.gz",\n  destfile = data_file,\n  mode = "wb"\n)\n\ntar(data_file, exdir = outdir)\ndata_dir <- file.path(outdir, "filtered_gene_bc_matrices", "hg19")\n\n#####\npbmc.data <- Read10X(data.dir = data_dir)\n\npbmc <- CreateSeuratObject(\n  counts = pbmc.data,\n  project = "pbmc3k",\n  min.cells = 3,\n  min.features = 200\n)\n```\n
```

- The **Seurat Object** is the central container that stores
  - single-cell expression data
  - all downstream analysis results (QC metrics, clustering, embeddings, markers).
- Here, we read in the counts and create the object
- **CreateSeuratObject** is the function used.

# Quality Control

```
``{r}
pbmc[["percent.mt"]] <- PercentageFeatureSet(pbmc, pattern = "^MT-")
head(pbmc[["percent.mt"]])
VlnPlot(pbmc, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"), ncol = 3)
``
```

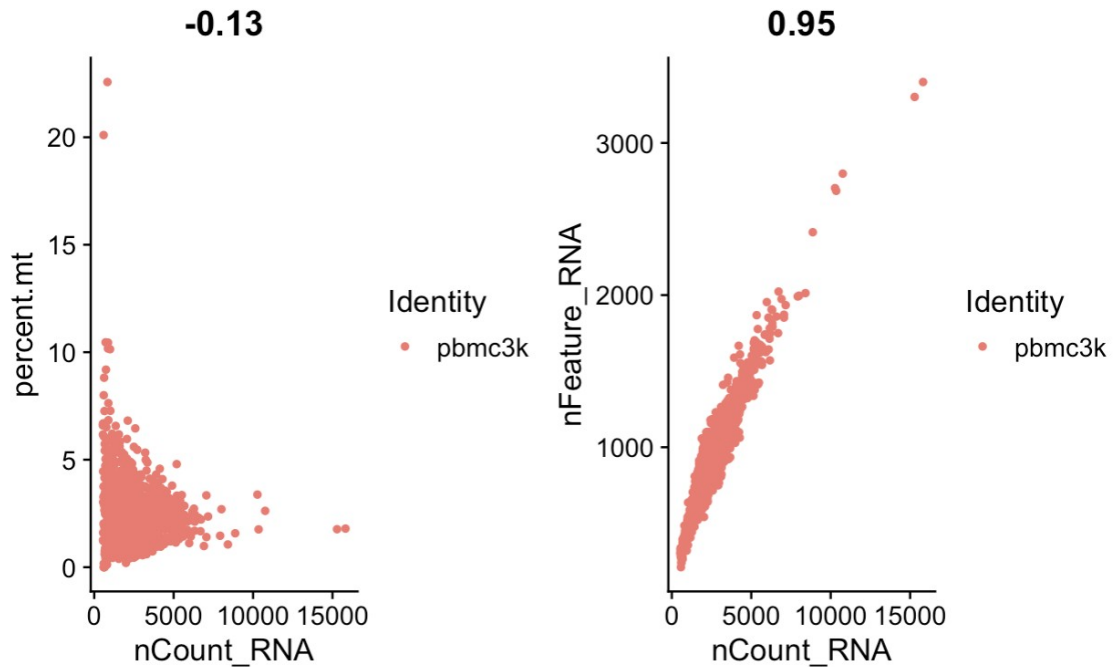
- Here, we add mitochondrial percentage as meta data
  - Note: Can use **double brackets** or **\$** to add to metadata
- The head() function gives the first few rows(cells)
- Seurat auto generates **nFeature\_RNA** and **nCount\_RNA** when creating the object
- High mitochondrial content could mean dying or low quality cells

# Practice Question

- Use `?VlnPlot` to get more information about the function
- Plot MALAT1 expression as a violin plot (no points shown).
  - Hint: Use the point size argument

# Quality Control

```
```{r}
plot1 <- FeatureScatter(pbm, feature1 = "nCount_RNA", feature2 = "percent.mt")
plot2 <- FeatureScatter(pbm, feature1 = "nCount_RNA", feature2 = "nFeature_RNA")
plot1 + plot2
```
```



- Checking relationships between quality metrics can help detect
  - Low quality cells
  - Doublets
  - Outliers
- This helps us decide cutoffs for filtering



# Filtering

```
``{r}
original_num_cells <- ncol(pbmc)
pbmc <- subset(pbmc, subset = nFeature_RNA > 200 & nFeature_RNA < 2500 & percent.mt < 5)
filtered_num_cells <- ncol(pbmc)
```

- The `subset` function is used to filter the cells
  - Number of genes expressed above 200
  - Number of genes expressed below 2500
  - Mitochondrial percentage < 5
- `ncol()` function will tell us how many cells are in the object
- Filtering is performed to remove **low quality cells** and **artifacts**

# Practice Question

- Create variable `pbmc2`, where we filter `pbmc` using these parameters
  - Number of genes expressed above 500
  - Number of genes expressed below 2000
  - Percent mitochondria < 4
- How many more cells are in `pbmc` than `pbmc2`?
- What percentage of the cells were filtered out?

# Solution

---

```
Filter to create pbmc2
pbmc2 <- subset(
 pbmc,
 subset = nFeature_RNA > 500 & nFeature_RNA < 2000 & percent.mt < 4
)

Number of cells before and after
num_pbmc <- ncol(pbmc)
num_pbmc2 <- ncol(pbmc2)

How many more cells are in pbmc than pbmc2
num_diff <- num_pbmc - num_pbmc2

Percent of cells filtered out
percent_filtered <- (num_diff / num_pbmc) * 100

Print results
num_diff
318

percent_filtered
11.77778
```

# Normalizing Data

```
```{r}
pbmc <- NormalizeData(pbmc, normalization.method = "LogNormalize", scale.factor = 10000)
```
```

- Counts need to be **normalized** so that **expression** can be compared **across cells**
  - **Sequencing depth** can differ between cells
  - **Highly expressed genes** can drown out biological signal
- Normalizing for sequencing depth:
  - Divide expression for all cells by their total counts and multiply by scale factor
- Reducing impact of highly expressed genes
  - Takes natural log of scale factored expression.
- This is all done with the **NormalizeData()** function

# Finding Variable Genes

```
```{r}
pbmc <- FindVariableFeatures(pbmc, selection.method = "vst", nfeatures = 2000)

# Identify the 10 most highly variable genes
top10 <- head(VariableFeatures(pbmc), 10)

# plot variable features with and without labels
plot1 <- VariableFeaturePlot(pbmc)
plot2 <- LabelPoints(plot = plot1, points = top10, repel = TRUE)
plot1 + plot2
```
```

- Highly variable genes capture biological differences strongly
- Analyzing only variable genes reduces noise and helps with dimensionality reduction.

# Scaling Data

```
```\r}  
all.genes <- rownames(pbmc)  
pbmc <- ScaleData(pbmc, features = all.genes)  
```
```

- ScaleData() function scales the data so that each gene's expression is on the same scale.
  - Centers expression for each gene at 0
  - Divides by standard deviation of that gene's expression
- This is important for dimensionality reduction so that each gene has equal importance.

# Dimensionality Reduction - PCA

```
```{r}  
pbmc <- RunPCA(pbmc, features = VariableFeatures(object = pbmc))  
```
```

- **PCA** reduces thousands of gene features into a few key components, making the data less noisy and easier to interpret.
  - Principal component: weighted combination of gene expression
  - PC1 explains the most variance in the data, PC2 explains the second most, etc.

# Visualizations with PCA

```
```{r}
VizDimLoadings(pbmcc, dims = 1:2, reduction = "pca")
```
```

```
```{r}
DimPlot(pbmcc, reduction = "pca") + NoLegend()
```
```

```
```{r}
DimHeatmap(pbmcc, dims = 1, cells = 500, balanced = TRUE)
```
```

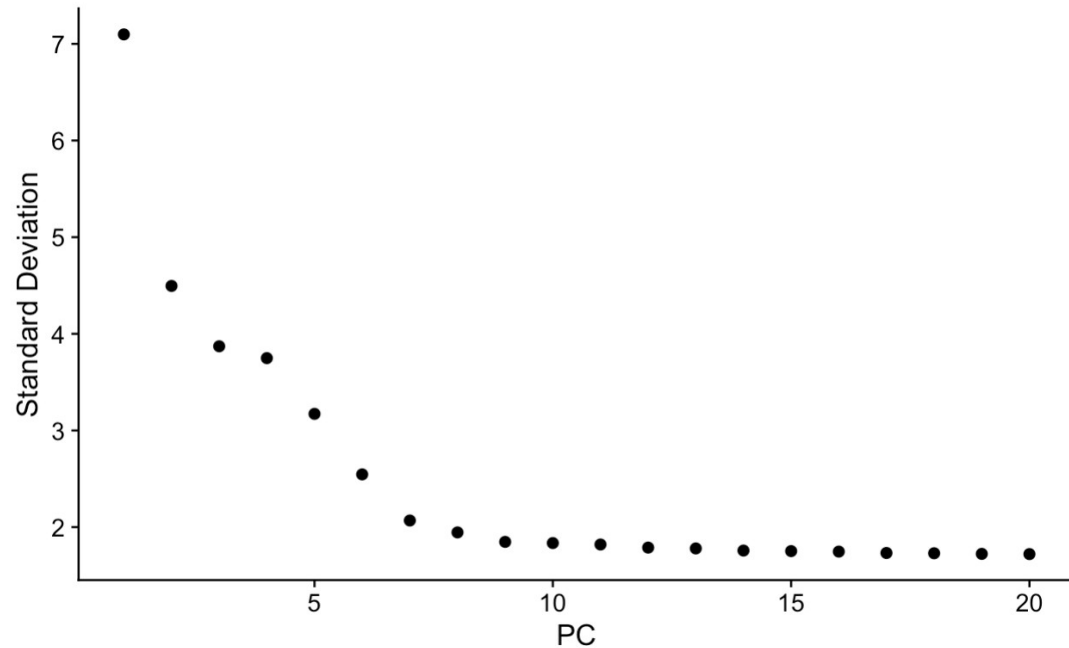
```
```{r}
ElbowPlot(pbmcc)
```
```

- **VizDimLoadings()** – Plots the loadings/weights for the top genes for each principal component you specify
- **DimPlot()** will plot the cells on an axis using two principal components
- **DimHeatmap()** creates a heatmap using top genes from a principal component
- **ElbowPlot()** Plots the variance explained by each PC



# Choosing how many PCs to use in downstream analysis

ElbowPlot(pbm)



- The elbow plot can help you decide how many PCs to use.
- There looks to be an elbow at around 10 PCs.
- Majority of the true signal is likely captured in these first 10 PCs

# Clustering the Cells

```
```{r}
pbmc <- FindNeighbors(pbmc, dims = 1:10)
pbmc <- FindClusters(pbmc, resolution = 0.5)
```
```

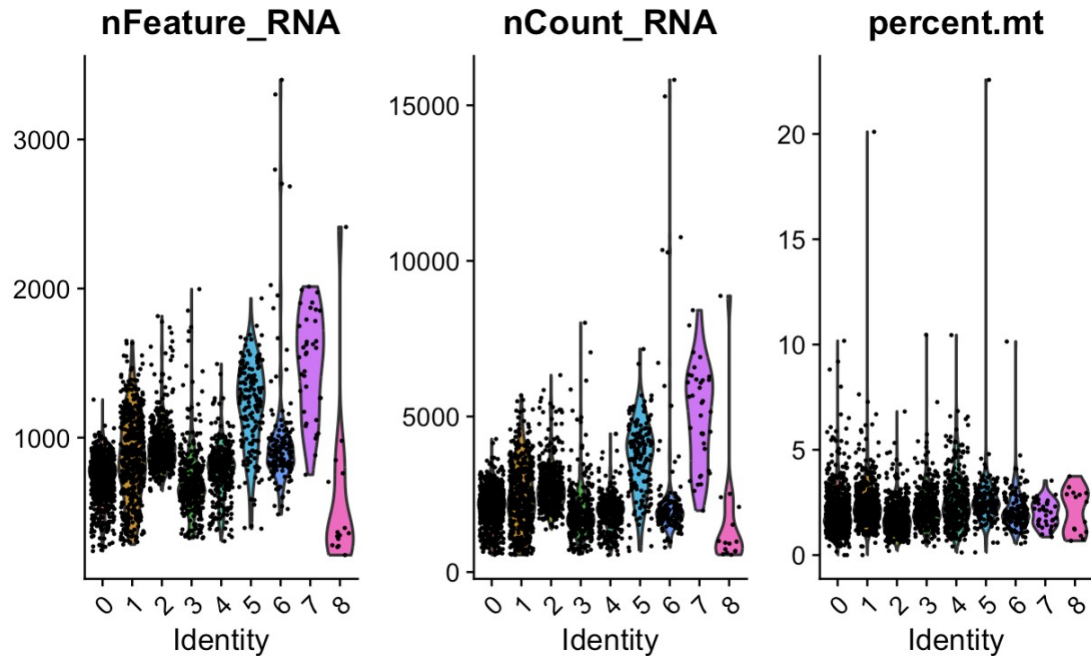
- **FindNeighbors()** builds a graph of cells based on how close they are in PCA space (who is nearest to who).
- **FindClusters()** then groups those cells into clusters the nearest neighbors graph.

# Clustering the Cells

```
```{r}
pbmc <- FindNeighbors(pbmc, dims = 1:10)
pbmc <- FindClusters(pbmc, resolution = 0.5)
```
```

- Note that with `FindNeighbors()`, you need to specify which PCs to use.
  - Based on the elbow plot, we will use PCs 1-10
- With `FindClusters`, you need to specify resolution.
  - Higher resolution gives more clusters
  - Lower resolution gives less clusters
- Number of clusters depends on study goals/shape of UMAP/known markers that should cluster separately

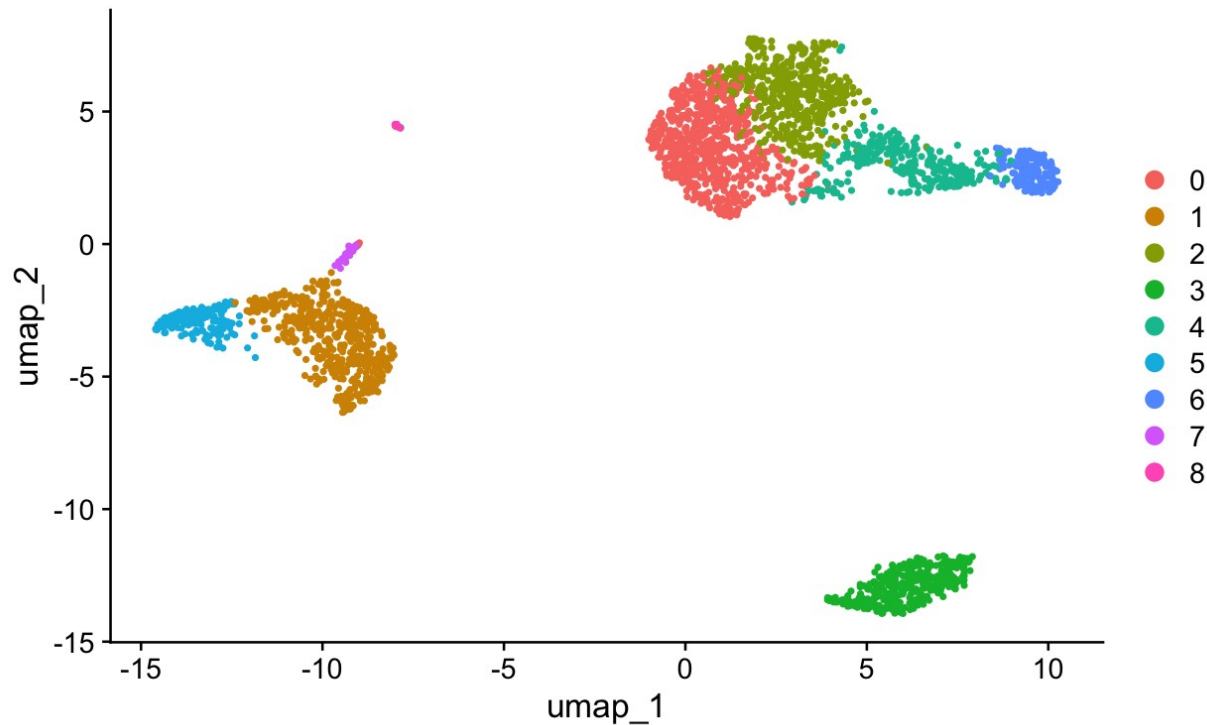
# Clustering the cells



- Clustering the cells will **change the active.ident** (identity) of the cells
- The violin plot we made before is now split between cluster identities
- `head(pbmc@active.ident)` can tell us what the active identity is

# UMAP – Non-linear dimensionality reduction

```
``{r}
pbmc <- RunUMAP(pbmc, dims = 1:10)
note that you can set `label = TRUE` or use the LabelClusters function to help label
individual clusters
DimPlot(pbmc, reduction = "umap")
``
```



- **UMAP** learns the structure of the data and projects it into 2D for visualization.
- It combines PCA-reduced information with neighbor relationships to preserve similarity between cells.

# RunUMAP – min.dist and n.neighbors parameters

## Usage

```
RunUMAP(object, ...)
```

```
Default S3 method:
```

```
RunUMAP(
 object,
 reduction.key = "UMAP_",
 assay = NULL,
 reduction.model = NULL,
 return.model = FALSE,
 umap.method = "uwot",
 n.neighbors = 30L,
 n.components = 2L,
 metric = "cosine",
 n.epochs = NULL,
 learning.rate = 1,
 min.dist = 0.3,
```

- **n.neighbors**: neighbors used for graph.
  - Small = local detail
  - Large = global structure.
- **min.dist**: spacing of points.
  - Small = tight clusters
  - Large = spread out clusters.

# Find Markers

```
```{r}
cluster2.markers <- FindMarkers(pbmc, ident.1 = 2)
head(cluster2.markers, n = 5)
```
```

R Console

data.frame  
5 x 5

Description: df [5 x 5]

|      | p_val<br><dbl> | avg_log2FC<br><dbl> | pct.1<br><dbl> | pct.2<br><dbl> | p_val_adj<br><dbl> |
|------|----------------|---------------------|----------------|----------------|--------------------|
| IL32 | 2.892340e-90   | 1.3070772           | 0.947          | 0.465          | 3.966555e-86       |
| LTB  | 1.060121e-86   | 1.3312674           | 0.981          | 0.643          | 1.453850e-82       |
| CD3D | 8.794641e-71   | 1.0597620           | 0.922          | 0.432          | 1.206097e-66       |
| IL7R | 3.516098e-68   | 1.4377848           | 0.750          | 0.326          | 4.821977e-64       |
| LDHB | 1.642480e-67   | 0.9911924           | 0.954          | 0.614          | 2.252497e-63       |

5 rows

```
```{r}
pbmc.markers <- FindAllMarkers(pbmc, only.pos = TRUE)

pbmc.markers %>%
  group_by(cluster) %>%
  dplyr::filter(avg_log2FC > 1)
```
```

- **FindMarkers** tells you which genes are differentially expressed in a certain cluster
- **FindAllMarkers** tells you which genes are differentially expressed in all clusters

R Console

grouped\_df  
7019 x 7

A tibble: 7,019 x 7

Groups: cluster [9]

|  | p_val<br><dbl> | avg_log2FC<br><dbl> | pct.1<br><dbl> | pct.2<br><dbl> | p_val_adj<br><dbl> | cluster<br><fctr> | gene<br><chr> |
|--|----------------|---------------------|----------------|----------------|--------------------|-------------------|---------------|
|  | 3.746131e-112  | 1.206019            | 0.912          | 0.592          | 5.137444e-108      | 0                 | LDHB          |
|  | 9.571984e-88   | 2.397366            | 0.447          | 0.108          | 1.312702e-83       | 0                 | CCR7          |
|  | 1.154695e-76   | 1.064113            | 0.845          | 0.406          | 1.583548e-72       | 0                 | CD3D          |

# Feature Plotting

```
```{r}
# you can plot raw counts as well
VlnPlot(pbmc, features = c("NKG7", "PF4"), slot = "counts", log = TRUE)
```
```

```
```{r}
FeaturePlot(pbmc, features = c("MS4A1", "GNLY", "CD3E", "CD14", "FCER1A", "FCGR3A", "LYZ", "PPBP",
"CD8A"))
```
```

```
```{r}
pbmc.markers %>%
  group_by(cluster) %>%
  dplyr::filter(avg_log2FC > 1) %>%
  slice_head(n = 10) %>%
  ungroup() -> top10

DoHeatmap(pbmc, features = top10$gene) + NoLegend()
```
```

- You can plot certain features on violin plots, UMAP, and heatmap.
  - Specify features = c("Gene1", "Gene2",...)



# Assigning Cell Types

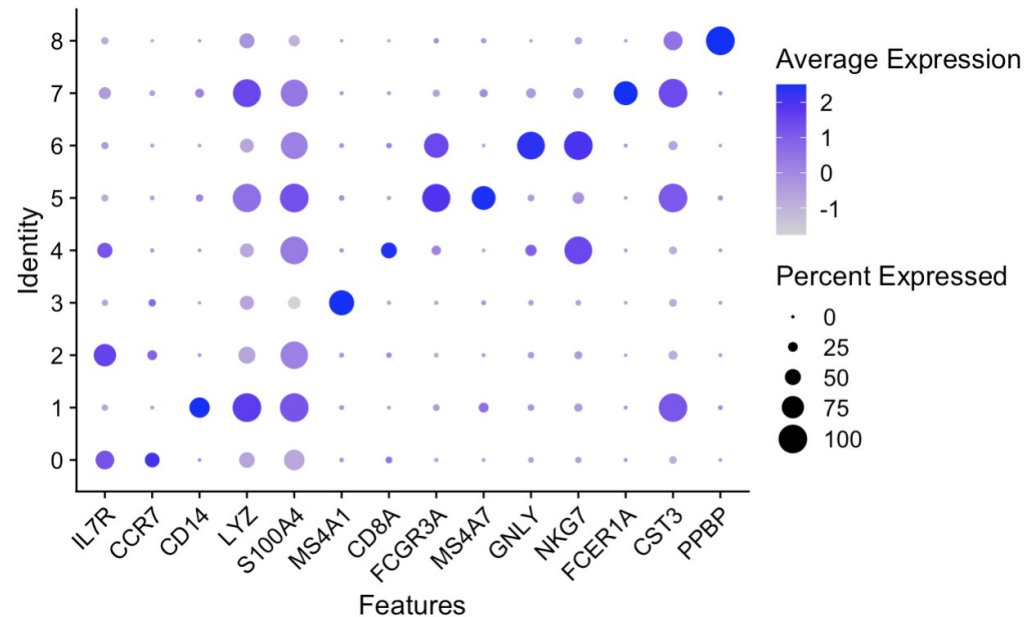
```

Define canonical markers by cluster
markers.to.plot <- c("IL7R", "CCR7", # Naive CD4+ T
 "CD14", "LYZ", # CD14+ Mono
 "S100A4", # Memory CD4+
 "MS4A1", # B
 "CD8A", # CD8+ T
 "FCGR3A", "MS4A7", # FCGR3A+ Mono
 "GNLY", "NKG7", # NK
 "FCER1A", "CST3", # DC
 "PPBP") # Platelet

Dot plot of marker expression across clusters
DotPlot(pbmc, features = markers.to.plot) +
 RotatedAxis()

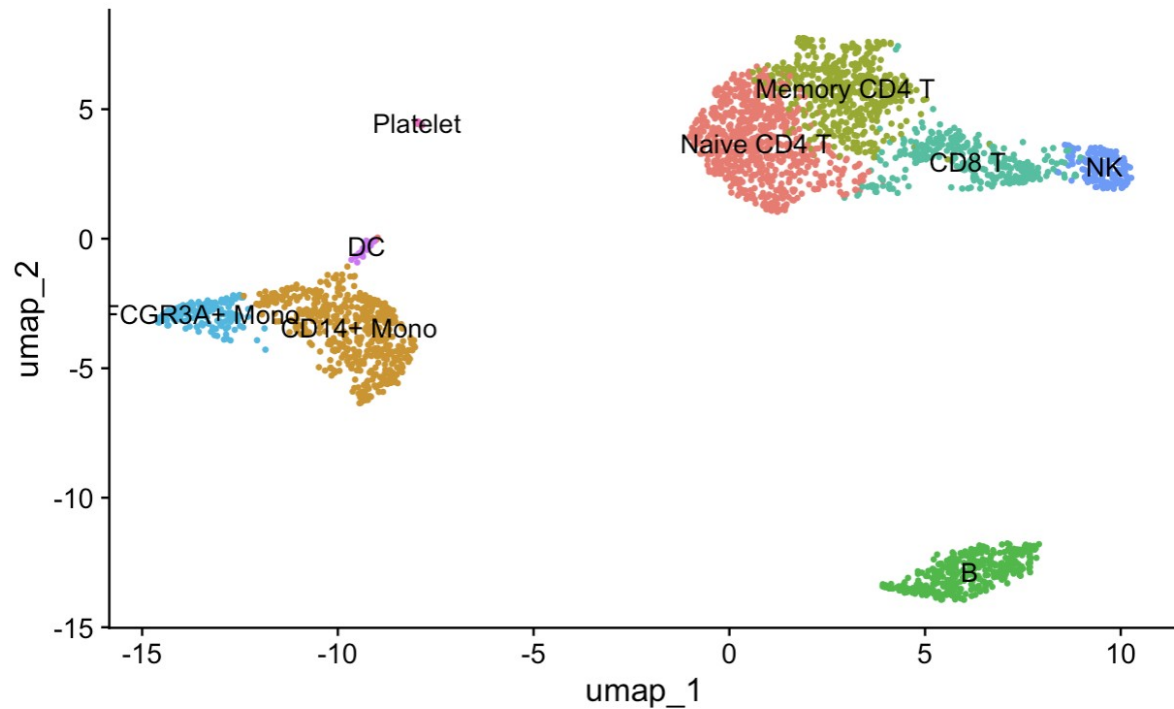
```

- Dot plots are a good way to visualize expression of pre-defined cell-type markers



# Assigning Cell Types

```
```{r}
new.cluster.ids <- c("Naive CD4 T", "CD14+ Mono", "Memory CD4 T", "B", "CD8 T", "FCGR3A+ Mono",
  "NK", "DC", "Platelet")
names(new.cluster.ids) <- levels(pbmc)
names(new.cluster.ids)
pbmc <- RenameIdents(pbmc, new.cluster.ids)
DimPlot(pbmc, reduction = "umap", label = TRUE, pt.size = 0.5) + NoLegend()
```
```



- The new cluster ids are stored as a list corresponding to cluster number.
- The names of the identity of the cells is then changed to reflect the cell type rather than the cluster number

# Saving/reading object

```
```{r}
saveRDS(pbmc, file = "~/single_cell_practice/pbmc3k_final_annotated.rds")

obj <- readRDS("~/single_cell_practice/pbmc3k_final_annotated.rds")
```
```

- R objects can be saved as “.rds” files.
- `saveRDS()` takes in an object and a path
- `readRDS()` will read in an object from a path



**Please scan the QR  
code to take our 1-  
minute survey!**

**Your feedback  
ensures we provide  
you with the best  
content**

